```
fs = require 'fs'
coffee = require 'coffee-script'

evalScruffyCoffeeFile = (fileName) ->
  fs.readFile fileName, 'utf-8', (error, scruffyCode) ->
    return if error
    tokens = coffee.tokens scruffyCode

    i = 0
    while token = tokens[i]
      isLambda = token[0] is 'IDENTIFIER' and /^λ[a-zA-Z]+$/.test token[1]
      if isLambda and tokens[i + 1][0] is '.'
        paramStart = ['PARAM_START', '(', {}]
        param = ['IDENTIFIER', token[1].replace(/λ/, ''), {}]
        paramEnd =  ['PARAM_END', ')', {}]
        arrow = ['->', '->', {}]
        indent = ['INDENT', 2, generated: true]
        tokens.splice i, 2, paramStart, param, paramEnd, arrow, indent

        j = i
        while tokens[j][0] isnt 'TERMINATOR'
          j++
        outdent = ['OUTDENT', 2, generated: true]
        tokens.splice j, 0, outdent
        i = i + 3
        continue
      i++
    nodes = coffee.nodes tokens
    javaScript = nodes.compile()
    eval javaScript

fileName = process.argv[2]
process.exit 'No file specified' unless fileName
evalScruffyCoffeeFile fileName
```

Loop through all the tokens. When a λ is encountered, the remainder of the tokens that make up the expression must be converted to an equivalent CoffeeScript function, including the correct indent levels.

Messing around with the token stream is difficult, potentially dangerous, and overall not a whole lot of fun. Luckily, there's a more structured representation of your CoffeeScript program that you can manipulate—the AST.

8.4.3. Using the abstract syntax tree

The next option available to you for implementing Scruffy's lambda syntax is to modify the abstract syntax tree in place before generating JavaScript. The AST is excellent for analyzing source code.

Modifying programs